

WINDOWS POWERSHELL

[Kaynak](#)

İÇERİK

1. What Is PowerShell (PowerShell Nedir?)
2. PowerShell Fundamentals: Başlangıç ve Temel Komutlar
3. Dosya Sistemi Gezintisi ve Yönetimi (File System Navigation)
4. Piping, Filtering ve Sorting (Veriyi İşlemek)
5. Sistem ve Ağ Bilgisi Toplama (System & Network Enumeration)
6. Gerçek Zamanlı Sistem Analizi (Real-Time System Analysis)
7. PowerShell Scripting ve Otomasyon

What Is PowerShell (PowerShell Nedir?)

Microsoft'un resmi tanımına göre: **"PowerShell, bir komut satırı kabuğu (shell), bir betik dili (scripting language) ve bir yapılandırma yönetimi çerçevesinden (configuration management framework) oluşan platformlar arası (cross-platform) bir görev otomasyon çözümüdür."**

Sadece siyah ekran deyip geçme; PowerShell, Windows sistem yönetiminin ve ileri seviye siber güvenlik operasyonlarının kalbidir.

Temel Özellikleri ve CMD'den Farkı

- **Altyapı:** .NET Framework üzerine inşa edilmiştir.
- **Nesne Yönelimli (Object-Oriented):** En kritik fark budur. Eski tip komut satırı araçları (CMD, Bash vb.) **metin (text)** tabanlıdır. PowerShell ise **nesne (object)** tabanlıdır.
 - **CMD:** Çıktı olarak sadece yazı verir. O yazıyı işlemek için metni kesip biçmen gerekir.
 - **PowerShell:** Çıktı olarak, içinde veri türleri ve yöntemler barındıran "canlı" nesneler verir.
- **Platform:** Eskiden sadece Windows'taydı, artık **macOS** ve **Linux** üzerinde de çalışıyor.

Kısa Bir Tarihçe: Neden İhtiyaç Duyuldu?

2000'lerin başında Windows, karmaşık kurumsal ortamlarda yaygınlaştıkça, eski dostumuz `cmd.exe` ve `.bat` dosyaları yetersiz kalmaya başladı. Sistem yöneticileri (SysAdmin) otomasyon yapmakta zorlanıyordu.

Jeffrey Snover (PowerShell'in Mimarı) Şunu Fark Etti:

- **Unix/Linux:** "Her şey bir dosyadır" (Everything is a file) mantığıyla çalışır. Metin işlemek kolaydır.
- **Windows:** Yapılandırılmış veri ve API'lar kullanır. Unix araçlarını Windows'a port etmek bu yüzden verimsizdi.

Snover'ın çözümü, .NET framework'ün gücünü betik (scripting) basitliğiyle birleştirmektir. 2006'da çıkan PowerShell, yöneticilerin **nesneleri manipüle ederek** Windows ile çok daha derin ve etkili konuşmasını sağladı.

Modern Dönem (2016 ve Sonrası): BT dünyası sadece Windows'tan ibaret değil. Microsoft buna **PowerShell Core** ile cevap verdi. Bu sürüm açık kaynaklıdır (open-source) ve Windows, macOS, Linux üzerinde çalışır. Yani öğrendiğin komutlar artık her yerde geçerli.

PowerShell'deki "Güç": Nesneler (Objects)

PowerShell'i anlamak için **Nesne (Object)** kavramını kafamıza kazımalıyız.

Programlamada Nesne Nedir? Bir nesne, özellikleri (**Properties**) ve yapabildiği eylemleri (**Methods**) olan bir öğedir.

- **Örnek (Araba):**
 - **Properties (Özellikler):** Renk , Model , YakıtSeviyesi (Durum bilgisi).
 - **Methods (Metotlar):** Sür() , KornaÇal() , YakıtDoldur() (Eylem/Fonksiyon).

PowerShell'de Nesne: PowerShell'de her şey bir nesnedir. Dosyalar, işlemler (processes), kullanıcı hesapları... Hepsi veri ve fonksiyonellik paketidir.

- Bir dosya nesnesi düşün:
 - **Properties:** Dosya adı, boyutu, oluşturulma tarihi.
 - **Methods:** Kopyala (Copy), Sil (Delete), Taşı (Move).

Geleneksel Shell vs. PowerShell (Cmdlet)

- **Geleneksel (CMD/Bash):** Komut çalışır -> Ekranı düz yazı (plain text) basar. Bu veriyi başka komuta sokmak için metni parse etmek (ayıklamak) gerekir.
- **PowerShell (Cmdlet):** PowerShell komutlarına **Cmdlet** (telaffuzu: *command-let*) denir. Bir cmdlet çalıştırıldığında düz yazı değil, **özellikleri ve metotları korunmuş bir nesne** döndürür.
 - *Avantajı:* Metni parçalamakla uğraşmazsın. Doğrudan nesnenin "Boyut" özelliğini isteyebilir veya "Sil" metodunu çağırabilirsin.

Özetle: PowerShell'de bir komutun çıktısı "görüntüden" ibaret değildir; o çıktı üzerinde doğrudan işlem yapabileceğin "canlı" bir veridir.

PowerShell Fundamentals: Başlangıç ve Temel Komutlar

Bu notlar, Windows sistemlerde PowerShell'in nasıl başlatılacağı, temel sözdizimi yapısı (Syntax), komut keşfi ve yardım dokümanlarının kullanımını kapsar. Özellikle sadece komut satırı erişimimiz olan (Reverse Shell vb.) durumlarda nasıl hareket edileceği detaylandırılmıştır.

1. PowerShell'i Başlatmak

PowerShell, duruma ve ortama göre farklı şekillerde başlatılabilir. GUI (Grafik Arayüz) erişimimiz varsa klasik yöntemler şunlardır:

- **Start Menu:** Arama çubuğuna `powershell` yazıp başlatmak.
- **Run Dialog (Win + R):** `powershell` yazıp Enter'a basmak.
- **File Explorer:** Herhangi bir klasörde adres çubuğuna `powershell` yazmak (O klasör konumunda terminal açar).
- **Task Manager:** File > Run new task > `powershell`.

CMD Üzerinden Başlatmak (Kritik Senaryo)

Sızma testlerinde (Pentest) veya CTF'lerde genellikle hedef makinede sadece **Command Prompt (cmd.exe)** erişimimiz olur. Bu durumda CMD içinden PowerShell oturumuna geçmek için şu komutu kullanırız:

PowerShell

```
C:\Users\captain> powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements!
https://aka.ms/PSWindowsPS C:\Users\captain>
```

Dikkat: Komutu çalıştırdıktan sonra satır başındaki prompt (istemi) `PS` ibaresiyle başlar (`PS C:\Users\captain>`). Bu, artık PowerShell ortamında olduğumuzu gösterir.

2. Temel Sözdizimi: Verb-Noun (Eylem-Nesne)

PowerShell komutlarına teknik olarak **Cmdlet** (telaffuzu: *command-lets*) denir. Geleneksel Windows komutlarından çok daha güçlüdürler.

Cmdlet'ler tutarlı bir isimlendirme standardına sahiptir: **Verb-Noun** (Eylem-Nesne).

- **Verb (Eylem):** Ne yapılacağını belirtir (Get, Set, New, Remove).
- **Noun (Nesne):** İşlemin neyin üzerinde yapılacağını belirtir (Content, Location, Item).

Örnekler:

- `Get-Content` : Bir dosyanın içeriğini **alır/getirir** (Linux'taki `cat` gibi).

- **Set-Location** : Mevcut çalışma dizinini **ayarlar/değiřtirir** (Linux'taki `cd` gibi).

3. Temel Cmdlet'ler ve Keřif

Hangi komutları kullanabileceğimizi bilmiyorsak veya mevcut oturumda nelerin yüklü olduğunu görmek istiyorsak `Get-Command` kullanırız.

Tüm Komutları Listeleme

PowerShell

```
PS C:\Users\captain> Get-Command
```

CommandType Source	Name	Version
----- -----	----	-----
Alias	Add-AppPackage	2.0.1.0
Appx		
Cmdlet	Add-AppxPackage	2.0.1.0
Appx		
Function	Add-BCDataCacheExtension	1.0.0.0
BranchCache		
...		

Çıktı Analizi:

- **CommandType**: Komutun türü (Alias, Function, Cmdlet).
- **Name**: Komutun adı.
- **Source**: Hangi modülden geldiğı.

Filtreleme (Önemli)

Çıktı çok uzun olduğu için belirli türdeki komutları filtrelemek isteyebiliriz. Örneğın, sadece **Function** türündeki komutları görmek için `-CommandType` parametresini kullanırız:

PowerShell

```
PS C:\Users\captain> Get-Command -CommandType "Function"
```

Bu mantık, ileride belirli anahtar kelimeleri içeren komutları ararken (örn: içinde "Network" geçenler) çok işimize yarayacak.

4. Get-Help: Nasıl Kullanılacağını Öğrenmek

Linux'taki `man` sayfaları neyse, PowerShell'de `Get-Help` odur. Bir cmdlet'in parametrelerini, kullanım şeklini ve örneklerini gösterir. **Ezberlemeye çalışma, `Get-Help` kullan.**

Örnek: `Get-Date` komutu hakkında bilgi almak:

PowerShell

```
PS C:\Users\captain> Get-Help Get-Date

NAME
    Get-Date

SYNOPSIS
    Gets the current date and time.

...

REMARKS
    To see the examples, type: "get-help Get-Date -examples".
    For more information, type: "get-help Get-Date -detailed".
    For technical information, type: "get-help Get-Date -full".
    For online help, type: "get-help Get-Date -online".
```

Kritik Parametreler: En alttaki **REMARKS** kısmı bize hayat kurtarıcı ipuçları verir:

1. `-examples` : Komutun nasıl kullanılacağına dair pratik örnekler sunar (En çok bunu kullanacaksın).
2. `-detailed` : Daha detaylı bilgi verir.
3. `-full` : Tüm teknik detayları döker.
4. `-online` : Tarayıcıda Microsoft'un dokümantasyonunu açar.

5. Aliases (Takma Adlar)

PowerShell, diğer kabuklardan (CMD veya Bash) gelen kullanıcıların zorluk çekmemesi için **Alias** (takma ad) sistemini kullanır.

Mevcut tüm takma adları görmek için `Get-Alias` kullanılır:

PowerShell

```
PS C:\Users\captain> Get-Alias

CommandType      Name                Definition
-----
Alias            cat -> Get-Content  (Dosya okuma)
Alias            cd -> Set-Location  (Dizin değiştirme)
Alias            dir -> Get-ChildItem (Dosya listeleme)
Alias            ls -> Get-ChildItem (Linuxçular için ls de çalışır)
```

Bu sayede `Get-ChildItem` yazmak yerine `dir` veya `ls` yazarak aynı işlemi yapabiliriz.

6. Modüller ve Harici Cmdlet'ler

PowerShell'in gücünü artırmak için internetteki depolardan (Repositories) yeni modüller indirilebilir. *Not: Bu işlem hedef makinede internet bağlantısı gerektirir. CTF ortamlarında (TryHackMe vb.) internet erişimi genellikle kapalıdır, bu yüzden bu komutlar hata verebilir.*

Modül Arama (Find-Module)

Modülün tam adını bilmiyorsak **Wildcard (*)** (joker karakter) kullanabiliriz. Örnek: Adı "PowerShell" ile başlayan modülleri aramak:

PowerShell

```
PS C:\Users\captain> Find-Module -Name "PowerShell*"
Version      Name              Repository      Description
-----
2.2.5        PowerShellGet     PSGallery       PowerShell module
with commands for discovering...
...
```

Modül Yükleme (Install-Module)

Bulduğumuz modülü yüklemek için:

PowerShell

```
PS C:\Users\captain> Install-Module -Name "PowerShellGet"
```

Güvenlik Uyarısı: Varsayılan olarak "Untrusted repository" (Güvenilmeyen depo) uyarısı alırsınız.

```
Are you sure you want to install the modules from 'PSGallery'?[Y] Yes ...
```

Burada **Y** diyerek onayı verip yüklemeyi tamamlarız. Bu sayede o modül içindeki yeni Cmdlet'ler kullanılabilir hale gelir.

Dosya Sistemi Gezintisi ve Yönetimi (File System Navigation)

PowerShell'de dosya sistemi üzerinde gezinmek (Navigation) ve dosya/klasör yönetimi yapmak için kullanılan Cmdlet'ler, klasik CMD veya Linux komutlarına benzer ancak çok daha standartlaştırılmış bir yapıdadır.

Sızma testlerinde sisteme sızdığımızda ilk işimiz genellikle **etrafı gezmek (Enumeration)** ve **kritik dosyaları okumaktır**. Bu bölüm o yüzden çok kritik.

1. Dizin ve Dosyaları Listeleme: Get-ChildItem

Bulduğumuz dizindeki veya belirlediğimiz bir yoldaki dosyaları listelemek için kullanılır.

- **CMD Karşılığı:** `dir`
- **Linux Karşılığı:** `ls`
- **Alias:** `dir`, `ls` (İkisi de çalışır).

Kullanım:

Eğer parametre vermezsek mevcut dizini listeler. Başka bir yeri listelemek için **-Path** parametresi kullanılır.

PowerShell

```
PS C:\Users\captain> Get-ChildItem
```

```
Directory: C:\Users\captain
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-r---	5/8/2021 9:15 AM		Desktop
d-r---	9/4/2024 10:58 AM		Documents
...			
d-----	5/8/2021 9:15 AM		Saved Games

Çıktı Analizi (Mode Sütunu):

Çıktıdaki `Mode` sütunu dosyanın türünü ve izinlerini gösterir:

- **d** (Directory): Bunun bir klasör olduğunu gösterir.
- **a** (Archive): Bunun bir dosya olduğunu gösterir.
- **r** (Read-only): Sadece okunabilir.
- **h** (Hidden): Gizli dosya (varsayılan çıktıda görünmeyebilir, `-Force` gerekebilir ama burada temel kullanımdayız).

2. Dizin Değiştirme: `Set-Location`

Mevcut çalışma dizinini değiştirmek (bir klasörün içine girmek veya çıkmak) için kullanılır.

- **CMD/Linux Karşılığı:** `cd`
- **Alias:** `cd`

Kullanım:

PowerShell

```
PS C:\Users\captain> Set-Location -Path ".\Documents"
PS C:\Users\captain\Documents>
```

Not: `.` (nokta) mevcut dizini temsil eder. `.\Documents` demek "şu an olduğum yerdeki Documents klasörüne git" demektir.

3. Dosya ve Klasör Oluşturma: `New-Item`

CMD'de klasör için `mkdir`, dosya için `type nul > dosya` gibi farklı komutlar varken; PowerShell'de **her şey bir "Item"dır**. Tek komutla hem dosya hem klasör oluşturulur, farkı `-ItemType` parametresi belirler.

Klasör Oluşturma

`-ItemType "Directory"` kullanırız.

PowerShell

```
PS C:\Users\captain\Documents> New-Item -Path ".\captain-cabin\captain-wardrobe" -ItemType "Directory"
```

Directory: C:\Users\captain\Documents\captain-cabin

Mode	LastWriteTime	Length	Name
d-----	9/4/2024 12:20 PM		captain-wardrobe

Burada iç içe (nested) bir yapı oluşturduk.

Dosya Oluşturma

`-ItemType "File"` kullanırız.

PowerShell

```
PS C:\Users\captain\Documents> New-Item -Path ".\captain-cabin\captain-wardrobe\captain-boots.txt" -ItemType "File"
```

Directory: C:\Users\captain\Documents\captain-cabin\captain-wardrobe

Mode	LastWriteTime	Length	Name
-a-----	9/4/2024 11:46 AM	0	captain-boots.txt

4. Silme İşlemi: `Remove-Item`

Tıpkı oluşturmada olduğu gibi, silmede de tek bir komut vardır. CMD'deki `del` (dosya) ve `rmdir` (klasör) ayrımı burada yoktur.

- **CMD/Linux Karşılığı:** `del`, `rm`
- **Alias:** `rm`, `del`

Kullanım:

Hem dosyayı hem de klasörü aynı şekilde sileriz.

PowerShell

```
# Önce dosyayı siliyoruz
PS C:\Users\captain\Documents> Remove-Item -Path ".\captain-cabin\captain-wardrobe\captain-boots.txt"

# Şimdi klasörü siliyoruz
PS C:\Users\captain\Documents> Remove-Item -Path ".\captain-cabin\captain-wardrobe"
```

Not: Dolu bir klasörü silmeye çalışırsanız PowerShell onay isteyebilir. Zorla silmek için `-Recurse` veya `-Force` parametreleri gerekebilir (İleride lazım olacak).

5. Kopyalama ve Taşıma

Dosyaları organize etmek veya (pentest senaryosunda) verileri bir "exfiltration" klasörüne toplamak için kullanılır.

Kopyalama: `Copy-Item`

- **Karşılığı:** `copy` (CMD), `cp` (Linux)
- Mantık: Kaynak -> Hedef

PowerShell

```
PS C:\Users\captain\Documents> Copy-Item -Path .\captain-cabin\captain-hat.txt -Destination .\captain-cabin\captain-hat2.txt
```

Taşıma: `Move-Item`

- **Karşılığı:** `move` (CMD), `mv` (Linux)
- Dosya ismini değiştirmek için de bu komut kullanılır (Rename mantığı).

6. Dosya İçeriğini Okuma: `Get-Content`

CTF'lerde ve sızma testlerinde en çok kullanacağımız komutlardan biri. `user.txt` veya `root.txt` bayraklarını (flags) okumak için bunu kullanacağız.

- **CMD Karşılığı:** `type`
- **Linux Karşılığı:** `cat`
- **Alias:** `cat` , `type`

Kullanım:

Dosyanın içeriğini terminal ekranına basar.

PowerShell

```
PS C:\Users\captain\Documents\captain-cabin> Get-Content -Path ".\captain-hat.txt"
```

```
Don't touch my hat!
```

Özet Tablo:

İşlem	PowerShell Cmdlet	Alias (Kısa Yol)	CMD	Linux
Listeleme	Get-ChildItem	dir , ls	dir	ls
Konum Değiştirme	Set-Location	cd	cd	cd
Oluşturma	New-Item	ni	mkdir , type	mkdir , touch
Silme	Remove-Item	rm , del	del , rmdir	rm
Kopyalama	Copy-Item	cp , copy	copy	cp
Taşıma	Move-Item	mv , move	move	mv
Okuma	Get-Content	cat , type	type	cat

Piping, Filtering ve Sorting (Veriyi İşlemek)

Bu bölüm, PowerShell'in gerçek gücünün ortaya çıktığı yerdir. Linux'taki Bash shell'ine aşina olanlar | (pipe) mantığını bilir ama PowerShell'de durum biraz farklı ve daha güçlü.

1. Piping Mantığı: Text vs Object

Linux/Bash ortamında bir komutun çıktısını diğerine | ile aktardığımızda, aslında saf **metin (text)** aktarırız. Ancak PowerShell'de pipe | sembolü **Nesne (Object)** taşır.

Neden Önemli?

Çünkü metin işlemede (string manipulation) veriyi parçalamak (cut, awk vb.) zordur. Ama nesne taşıdığımızda, verinin özellikleri (Property) ve metodları da beraberinde gelir.

- Örneğin bir dosya sadece "isimden" ibaret bir yazı değil; Length (boyut), LastWriteTime (değiştirilme tarihi), Extension (uzantı) gibi özellikleri olan bir nesnedir.

2. Sıralama: Sort-Object

Verileri belirli bir özelliğe göre sıralamak için kullanılır. `Get-ChildItem` ham veriyi getirir, biz bunu pipe ile `Sort-Object` 'e atarız.

Örnek: Dosyaları boyutuna (`Length`) göre sıralamak:

PowerShell

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Sort-Object Length
```

Directory: C:\Users\captain\Documents\captain-cabin

Mode	LastWriteTime	Length	Name
-a----	9/4/2024 12:50 PM	0	captain-boots.txt
-a----	9/4/2024 12:14 PM	264	captain-hat2.txt
...			
-a----	9/4/2024 12:37 PM	2116	ship-flag.txt

Görüldüğü gibi, 0 byte'tan başlayıp artarak sıraladı.

3. Filtreleme: `Where-Object`

Gelen verinin hepsini değil, sadece belirli kriterlere uyanları almak istiyorsak kullanılır. SQL'deki `WHERE` şartı gibidir.

Örnek: Sadece `.txt` uzantılı dosyaları getirmek:

PowerShell

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Where-Object -Property "Extension" -eq ".txt"
```

Karşılaştırma Operatörleri (Çok Önemli)

PowerShell'de `=`, `>`, `<` gibi işaretler yerine (Bash ve Python benzeri) özel operatörler kullanılır. Bunları adımız gibi bilmeliyiz:

Operatör	Anlamı	Açıklama
<code>-eq</code>	Equal	Eşittir.
<code>-ne</code>	Not Equal	Eşit değildir.
<code>-gt</code>	Greater Than	Büyüktür.
<code>-ge</code>	Greater/Equal	Büyük veya eşittir.
<code>-lt</code>	Less Than	Küçüktür.
<code>-le</code>	Less/Equal	Küçük veya eşittir.

Operatör	Anlamı	Açıklama
-like	Like	Desen (Pattern) eşleştirme (Wildcard * ile kullanılır).

Wildcard ile Filtreleme Örneği:

İsmi "ship" ile başlayan dosyaları bulmak (-like kullanımı):

PowerShell

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Where-Object -Property "Name" -like "ship*"
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	9/4/2024 12:37 PM	2116	ship-flag.txt

4. Özellik Seçme: Select-Object

Bazen bir nesnenin yüzlerce özelliği olabilir ama biz sadece ismini ve boyutunu görmek isteriz. Where-Object **satırları** filtrelerken (hangi dosyalar gelsin), Select-Object **sütunları** seçer (dosyanın hangi bilgisini göreyim).

Örnek: Sadece dosya adı ve boyutunu görmek:

PowerShell

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Select-Object Name, Length
```

Name	Length
----	-----
captain-wardrobe	
captain-boots.txt	0
captain-hat.txt	264
...	

5. Zincirleme (Chaining) - Combo Yapmak

Pipe'lar sadece iki komutla sınırlı değildir. Birden fazla komutu ardışık bağlayabiliriz.

Senaryo: Klasördeki **en büyük dosyayı** bulmak istiyoruz.

Mantık:

1. Dosyaları al (Get-ChildItem)
2. Boyuta göre **tersten** sırala (Sort-Object Length -Descending) -> En büyük en başa gelir.

3. İlk sıradakini seç (Select-Object -First 1)

PowerShell

```
PS C:\Users\captain> Get-ChildItem | Sort-Object Length -Descending |  
Select-Object -First 1
```

Directory: C:\Users\captain\Documents\captain-cabin

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	9/4/2024 12:37 PM	2116	ship-flag.txt

Bu komut zinciri, manuel arama yapmaktan çok daha hızlı ve hatasızdır.

6. Dosya İçinde Arama: Select-String

Dosyaların içeriğinde belli bir metni (text pattern) aramak için kullanılır. Log incelemelerinde veya şifre aramalarında hayat kurtarır.

- **Linux Karşılığı:** grep
- **Windows CMD Karşılığı:** findstr

Örnek: "captain-hat.txt" dosyasında "hat" kelimesini aramak:

PowerShell

```
PS C:\Users\captain\Documents\captain-cabin> Select-String -Path ".\captain-  
hat.txt" -Pattern "hat"
```

```
captain-hat.txt:8:Don't touch my hat!
```

Çıktı bize dosya adını, satır numarasını (:8:) ve eşleşen satırı gösterir.

Not: Select-String tam Regex (Regular Expression) desteğine sahiptir. Yani çok karmaşık desenleri (örn: email formatı, IP adresi formatı) de arayabiliriz.

Sistem ve Ağ Bilgisi Toplama (System & Network Enumeration)

Bir sisteme sızdıktan veya yönetici olarak giriş yaptıktan sonra ilk adım **Durumsal Farkındalık (Situational Awareness)** kazanmaktır. "Ben neredeyim?", "Bu makine ne?", "Ağda kiminle konuşuyor?" sorularının cevabını PowerShell ile CMD'den çok daha detaylı alabiliriz.

Bu bölüm, sistem yöneticileri (ve biz siber güvenlikçiler) için kritik olan **System Enumeration** (Sistem Keşfi) komutlarını kapsar.

1. Detaylı Sistem Bilgisi: Get-ComputerInfo

Eskiden CMD'de `systeminfo` kullanırdık. PowerShell'de bunun karşılığı `Get-ComputerInfo` 'dur, ancak çok daha güçlüdür. İşletim sistemi versiyonu, BIOS detayları, yama seviyeleri (hotfixes) gibi **tüm sistem konfigürasyonunun anlık görüntüsünü (snapshot)** tek komutla nesne olarak döndürür.

PowerShell

```
PS C:\Users\captain> Get-ComputerInfo

WindowsBuildLabEx           :
20348.859.amd64fre.fe_release_svc_prod2.220707-1832
WindowsCurrentVersion       : 6.3
WindowsEditionId            : ServerDatacenter
WindowsInstallationType     : Server Core
WindowsInstallDateFromRegistry : 4/23/2024 6:36:29 PM
WindowsProductId            : 00454-60000-00001-AA763
WindowsProductName          : Windows Server 2022 Datacenter
[...]
```

Çıktı Analizi (Pentest Gözüyle):

- **WindowsEditionId:** `ServerDatacenter` . Hedefin bir sunucu olduğunu anlıyoruz.
- **WindowsInstallationType:** `Server Core` . Grafik arayüzü (GUI) olmayan, sadece komut satırı ile yönetilen bir sunucu. Bu, saldırı yüzeyinin daha dar olduğunu ama komut satırına hakim olmamız gerektiğini gösterir.
- **WindowsProductName:** Tam sürüm bilgisi (Server 2022). Exploit ararken bu versiyona göre arama yapacağız.

2. Kullanıcı Hesapları: Get-LocalUser

Makinede hangi yerel kullanıcıların olduğunu görmek, güvenlik yapılandırmasını anlamak ve yetki yükseltme (Privilege Escalation) fırsatlarını kollamak için kullanılır.

- **CMD Karşılığı:** `net user`

PowerShell

```
PS C:\Users\captain> Get-LocalUser

Name                Enabled Description
-----
Administrator      True    Built-in account for administering the
computer/domain
captain              True    The beloved captain of this pirate ship.
DefaultAccount      False   A user account managed by the system.
```

Guest	False	Built-in account for guest access to the computer/domain
WDAGUtilityAccount	False	A user account managed and used by the system...

Kritik Kolonlar:

- **Name:** Kullanıcı adı.
- **Enabled:** Hesabın aktif olup olmadığı (True / False). False olan hesaplarla (örneğin Guest) genellikle vakit kaybetmeyiz.
- **Description:** Burası bazen altın madenidir. Tembel sistem yöneticileri buraya "Yedek admin şifresi: 123456" gibi notlar bırakabilir veya hesabın ne amaçla kullanıldığını yazar.

3. Ağ Yapılandırması (Network Configuration)

CMD'deki `ipconfig` komutunun modern ve detaylı versiyonlarıdır. İki temel Cmdlet kullanırız:

A. Genel Bakış: `Get-NetIPConfiguration`

Sistemin ağ arayüzlerini, IP adreslerini, DNS sunucularını ve Gateway (Ağ Geçidi) bilgilerini özet olarak verir. Hızlı bir kontrol için idealdir.

PowerShell

```
PS C:\Users\captain> Get-NetIPConfiguration

InterfaceAlias      : Ethernet
InterfaceIndex      : 5
InterfaceDescription : Amazon Elastic Network Adapter
IPv4Address          : 10.10.178.209
IPv4DefaultGateway  : 10.10.0.1
DNSServer            : 10.0.0.2
```

- **InterfaceIndex (5):** Bu numara (Index ID), ileride rota (route) eklerken veya spesifik arayüzde işlem yaparken lazım olacak.
- **IPv4Address:** Makinenin ağdaki kimliği.

B. Derinlemesine Bakış: `Get-NetIPAddress`

Sadece aktif olanları değil, **tüm** yapılandırılmış IP adreslerini (IPv4 ve IPv6 dahil) ve bunların durumlarını listeler. Loopback (127.0.0.1) arayüzleri dahil her şeyi döker.

PowerShell

```
PS C:\Users\captain> Get-NetIPAddress

IPAddress      : fe80::3fef:360c:304:64e%5
```

```
InterfaceIndex      : 5
AddressFamily       : IPv6
AddressState        : Preferred
...
IPAddress           : 10.10.178.209
InterfaceIndex      : 5
InterfaceAlias       : Ethernet
AddressFamily       : IPv4
...
```

Neden Get-NetIPAddress Kullanırız?

- **AddressFamily:** IPv4 mü IPv6 mı olduğunu netleştirir.
- **AddressState:** Adresin "Preferred" (Tercih edilen/Aktif) durumda olup olmadığını gösterir.
- **PrefixLength:** Subnet mask bilgisini (örn: /24 yerine 24 olarak) verir.
- Gizli veya kullanılmayan IP bloklarını görmek için `Get-NetIPConfiguration` yetersiz kaldığında buna başvurulur.

Gerçek Zamanlı Sistem Analizi (Real-Time System Analysis)

Statik sistem bilgilerini (OS versiyonu, IP adresi vb.) aldıktan sonra, asıl olay **şu an makinede neler döndüğünü** anlamaktır. Bu bölüm, sistemde o an çalışan süreçleri, servisleri ve aktif ağ bağlantılarını incelemek için kullanılır. **Olay Müdahalesi (Incident Response)** ve **Tehdit Avcılığı (Threat Hunting)** senaryolarında saldırıyı yakalamak için en kritik komutlar buradadır.

1. Çalışan Süreçleri İzleme: Get-Process

Sistemdeki RAM ve CPU kaynaklarını tüketen aktif programları listeler.

- **Linux Karşılığı:** `ps` , `top`
- **Kullanım Amacı:** Anormal CPU kullanımı olan süreçleri veya ismi şüpheli (örn: `mimikatz.exe` veya kendini gizlemeye çalışan `svchostt.exe`) zararlıları tespit etmek.

PowerShell

```
PS C:\Users\captain> Get-Process
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
-----	-----	-----	-----	-----	--	--	-----
67	5	872	500	0.06	2340	0	AggregatorHost
55	5	712	2672	0.02	3024	0	AM_Delta_Patch...
309	13	18312	1256	0.52	1524	0	amazon-ssm-agent
...							

Çıktı Analizi:

- **Id (PID):** Process ID. Bu numara çok önemlidir. Birazdan ağ bağlantılarını incelerken hangi bağlantının hangi programa ait olduğunu bu ID ile eşleştireceğiz.
- **ProcessName:** Uygulamanın adı.
- **CPU(s) & WS(K):** Kaynak tüketimi. Bir zararlı (miner vb.) çalışıyorsa burada tavan yapar.

2. Servis Durumları: Get-Service

Arka planda çalışan veya durdurulmuş servisleri listeler. Sistem yöneticileri bunu sorun gidermek için kullanır ama biz güvenlikçiler **Persistence (Kalıcılık)** mekanizmalarını aramak için kullanırız. Saldırganlar genellikle makine yeniden başladığında erişimi kaybetmemek için kendi zararlılarını bir "Servis" olarak kaydederler.

PowerShell

```
PS C:\Users\captain> Get-Service
```

Status	Name	DisplayName
-----	----	-----
Stopped	Amazon EC2Launch	Amazon EC2Launch
Running	AmazonSSMAgent	Amazon SSM Agent
Stopped	AppIDSvc	Application Identity
Running	BFE	Base Filtering Engine
...		

- **Status:** Running (Çalışıyor) veya Stopped (Durdu).
- **Name:** Servisin kısa adı.

3. Ağ Bağlantılarını İnceleme: Get-NetTCPConnection

Belki de en kritik komut budur. Bilgisayarın şu an **kiminle konuştuğunu** gösterir. Saldırganın komuta kontrol (C2) sunucusuna giden bağlantıları veya içeriye açtığı arka kapıları (backdoor) burada yakalarız.

- **Eski Karşılığı:** netstat -ano

PowerShell

```
PS C:\Users\captain> Get-NetTCPConnection
```

LocalAddress	LocalPort	RemoteAddress	RemotePort	State	OwningProcess
-----	-----	-----	-----	-----	-----
0.0.0.0	3389	0.0.0.0	0	Listen	980

```
10.10.178.209 22 10.14.87.60 53523 Established 1444
...
```

Kritik Parametreler ve Analiz:

1. **LocalPort:** Hangi portun açık olduğu. (Örn: 3389 RDP açık, 22 SSH açık).
2. **RemoteAddress:** Bağlantının karşı ucu. Eğer burada bilmediğimiz bir IP (örneğin Rusya veya Çin IP'si) varsa alarm zilleri çalar.
3. **State:**
 - **Listen:** Bağlantı bekliyor (Port açık).
 - **Established:** Aktif bağlantı var (Veri alışverişi yapılıyor). **En tehlikelisi budur.**
4. **OwningProcess:** Bu bağlantıyı hangi PID kullanıyor? (Yukarıdaki `Get-Process` ile bu ID'ye bakıp hangi `exe` olduğunu buluruz).

4. Dosya Bütünlüğü ve Hash Kontrolü: `Get-FileHash`

Olay müdahalesinde (Incident Response), bir dosyanın değiştirilip değiştirilmediğini (tampering) anlamak veya elimizdeki şüpheli dosyanın bilinen bir virüs olup olmadığını (VirusTotal'de aratmak için) anlamak amacıyla hash değeri alınır.

PowerShell

```
PS C:\Users\captain\Documents\captain-cabin> Get-FileHash -Path .\ship-flag.txt
```

Algorithm	Hash	Path
-----	----	----
SHA256	54D2EC3C12BF3D...	C:\...\ship-flag.txt

- Varsayılan algoritma **SHA256**'dır. Başka algoritma gerekirse `-Algorithm MD5` gibi parametre eklenebilir.

5. Gizli Veri Akışları (Alternate Data Streams - ADS)

Windows NTFS dosya sisteminde, bir dosyanın arkasına gizlenmiş başka veriler olabilir. Buna **ADS** denir. Dosya boyutu (Explorer'da görünen) değişmez ama arkada veri saklanır. Saldırganlar payload'larını buraya gizleyebilir.

Normal `Get-ChildItem` veya `ls` komutu bunları **GÖSTERMEZ**. Görmek için `Get-Item` komutuna `-Stream *` parametresi eklenmelidir.

(Not: Aşağıdaki örnek eğitim amaçlıdır, VM'de bu dosya olmayabilir ama mantık aynıdır.)

PowerShell

```
PS C:\Users\Alex\Documents> Get-Item -Path "C:\House\house_log.txt" -Stream *
```

```
FileName      : C:\House\house_log.txt
Stream        : :$DATA      <-- Bu dosyanın normal içeriğidir.
Length       : 13
```

```
FileName      : C:\House\house_log.txt
Stream        : housinginfo <-- BU BİR ADS'DİR! Dosyaya eklenmiş gizli veri.
Length       : 21
```

Analiz:

- `:$DATA` : Standart dosya içeriği akışı. Her dosyada vardır.
- `housinginfo` (veya başka bir isim): Dosyaya sonradan eklenmiş gizli akış. Eğer sistem dosyalarında (örn: `calc.exe:payload`) böyle bir şey görürsek, makine %99 hacklenmiştir.

PowerShell Scripting ve Otomasyon

Bu bölüm, PowerShell'in sadece tek satırlık komutlardan ibaret olmadığını, bir "programlama dili" gibi kullanılarak karmaşık işlerin nasıl otomatize edildiğini anlatıyor. Özellikle **Blue Team** ve **Red Team** operasyonlarında manuel hımmallıktan kurtulmak ve ölçeklenebilir işler yapmak için scripting şart.

1. Scripting Nedir?

Scripting, bilgisayara bir "yapılacaklar listesi" (To-Do List) vermektir. Manuel olarak tek tek yazdığımız komutları bir metin dosyasına (`.ps1` uzantılı) kaydederiz ve bilgisayar bunları sırasıyla, hatasız ve hızlı bir şekilde çalıştırır.

Neden Kritik?

- **Blue Team (Savunma)**: Log analizi, anormallik tespiti ve IOC (Indication of Compromise) taramalarını otomatize etmek için.
- **Red Team (Saldırı)**: Enumeration (keşif), payload çalıştırma, AV/EDR atlatma (obfuscation) ve yetki yükseltme işlemlerini hızlandırmak için.
- **SysAdmin**: Yüzlerce sunucunun konfigürasyonunu tek seferde yönetmek için.

2. Uzaktan Komut Çalıştırma: `Invoke-Command` (Çok Önemli)

Pentestlerde ve sistem yönetiminde en sık kullanacağımız, "Lateral Movement" (Yanal Hareket) veya toplu yönetim için kritik olan Cmdlet: **Invoke-Command**.

Bu komut, yerel makineden (kendi bilgisayarımızdan) uzaktaki bir veya birden fazla bilgisayarda komut çalıştırmamızı sağlar.

Temel Mantık: Ben buradayım -> Komutu gönderiyorum -> Karşı sunucuda çalışıyor -> Çıktısı bana dönüyor.

`Get-Help Invoke-Command -examples` çıktısına göre iki ana kullanım senaryosu vardır:

A. Yerel Scripti Uzakta Çalıştırmak (`-FilePath`)

Elimizde hazır bir script var (`test.ps1`) ve bunu hedef sunucuda (`Server01`) çalıştırmak istiyoruz. Script dosyasının karşı tarafta olmasına gerek yok, bizden oraya gidip çalışır.

PowerShell

```
Invoke-Command -FilePath c:\scripts\test.ps1 -ComputerName Server01
```

- **-FilePath:** Yerel bilgisayardaki scriptin yolu.
- **-ComputerName:** Hedef bilgisayarın adı veya IP adresi.
- **Mantık:** Script yerelde durur, işlem uzakta yapılır, sonuç yerel ekrana basılır.

B. Komut Bloğu Çalıştırmak (`-ScriptBlock`)

Dosya ile uğraşmak istemiyorsak veya tek seferlik bir komut (One-liner) çalıştıracaksak bu yöntemi kullanırız. Pentest sırasında en pratik yöntem budur.

PowerShell

```
Invoke-Command -ComputerName Server01 -Credential Domain01\User01 -  
ScriptBlock { Get-Culture }
```

- **-ComputerName:** Hedef sunucu (`Server01`).
- **-Credential:** Komutu hangi yetkiyle çalıştıracağız? (Burada `Domain01\User01` kullanıcısı belirtilmiş. PowerShell parolanızı soran bir pencere açar).
- **-ScriptBlock { ... }:** Süslü parantez içine yazılan her şey karşı tarafta çalışır. Buraya `Get-Process` , `ipconfig` veya zararlı bir payload yazabiliriz.
- **Sonuç:** Sanki o makinenin başına geçmişiz gibi komut orada çalışır ve çıktısı bize gelir.

Özet: Eğer bir script dosyanız varsa **-FilePath**, anlık komut yazacaksanız **-ScriptBlock** parametresini kullanın. `Invoke-Command` PowerShell'in "SSH"ı gibidir, çok güçlüdür.